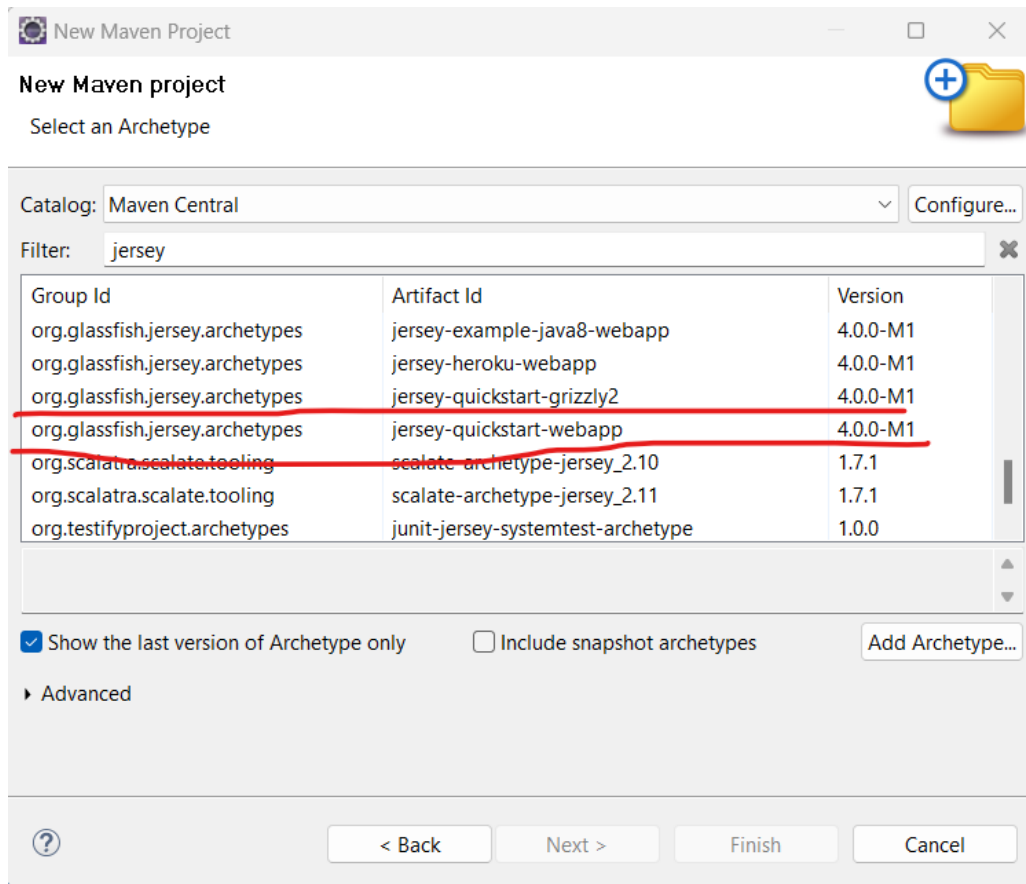
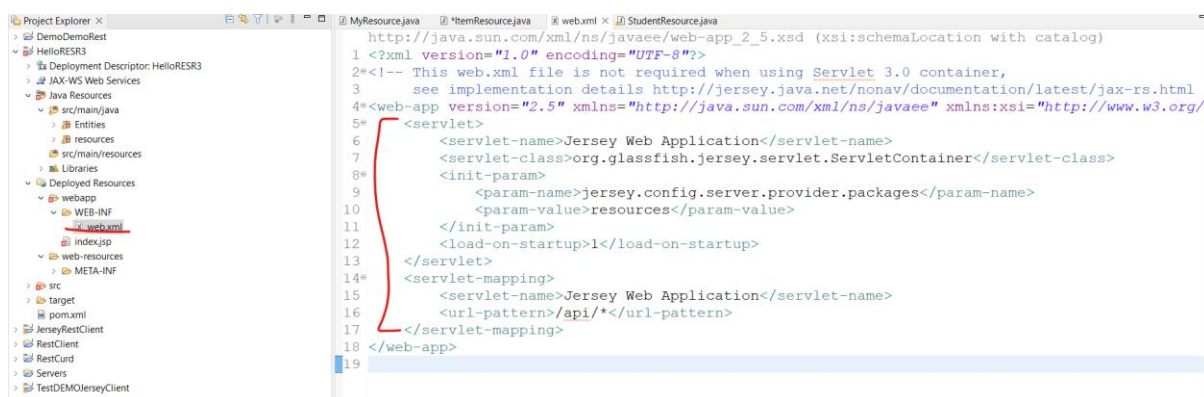


Activity: Creating Web Services using Jersey

Step 1: Create a Maven Project by using following archetype



Step2. Find web.xml under WEB-INF and configure Servlet



Code:

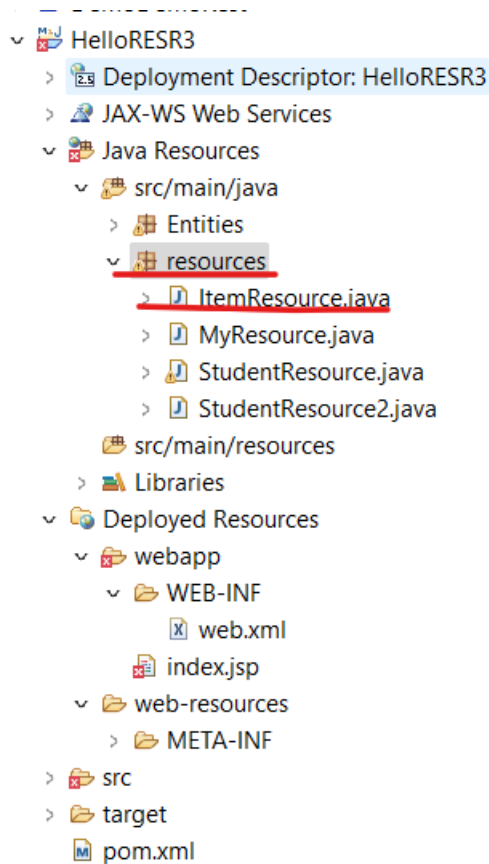
```
<servlet>
  <servlet-name>Jersey Web Application</servlet-name>
  <servlet-
class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
  <init-param>
```

```

        <param-
name>jersey.config.server.provider.packages</param-name>
        <param-value>resources</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>Jersey Web Application</servlet-name>
    <url-pattern>/api/*</url-pattern>
</servlet-mapping>

```

Step3. Create resources package under src/main/java and under this package create your resource classes



Step4. Create resource class with constructor and `@PostConstruct` to initialize object after Bean Loading

```
@Path("/items")
public class ItemResource {

    List<Item> itemList;

    public ItemResource() {
        this.itemList = new ArrayList<Item>(); // Initialize itemList
    }
    @PostConstruct
    public void init() {
        // Add initial items
        itemList.add(new Item(1, "Item One"));
        itemList.add(new Item(2, "Item Two"));
    }
}
```

Explanation:

We defined a JAX-RS resource class named `ItemResource` that represents a collection of items.

Here's a breakdown of what each part does:

1. `@Path("/items")` Annotation:

- This annotation is used on the class declaration and plays a crucial role in how clients access resources through URIs.
- In this case, it specifies the base URI path for this resource as `/items`.
- When a client sends a request to a URI starting with `/items`, Jersey will look for a resource class annotated with `@Path("/items")` to handle the request.

2. `ItemResource` Class:

- This class acts as the resource class responsible for managing and providing access to items.
- It likely represents a collection of items or an endpoint for CRUD (Create, Read, Update, Delete) operations on items.

3. `List<Item> itemList;`

- This line declares a member variable named `itemList` of type `List<Item>`.
- It will hold a collection of `Item` objects, which presumably represent the items this resource manages.

4. `public ItemResource()` Constructor:

- This is the default constructor for the `ItemResource` class.
- In this case, it initializes the `itemList` with a new `ArrayList<Item>()`.
- This ensures that the `itemList` is always initialized, even if `init()` is not called explicitly.

5. `@PostConstruct` Annotated Method:

- The `@PostConstruct` annotation is used on a method to indicate that it should be invoked after the constructor is executed and dependencies are injected (if any).
- The `init()` method here is likely used to populate the `itemList` with some initial items.
- It creates two `Item` objects and adds them to the list.

Step5. Create Method to handle Post request

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response createItem(Item item) {
    //item.setId(itemList.size() + 1);
    //itemList.add(item);
    System.out.println(item);
    return Response.status(Response.Status.CREATED)
        .entity("Item created successfully").build();
}
```

Explanation:

The Code snippet defines a JAX-RS resource method named `createItem` within the `ItemResource` class.

Let's break down what it does:

Annotations:

- `@POST`: This annotation indicates that this method handles HTTP POST requests. Typically used for creating new resources.
- `@Consumes(MediaType.APPLICATION_JSON)`: This annotation specifies that the method expects JSON data in the request body. It tells Jersey that the method can handle requests with the `Content-Type` header set to `application/json`.
- `@Produces(MediaType.APPLICATION_JSON)`: This annotation indicates that the method will produce a response in JSON format. It tells Jersey that the response will have the `Content-Type` header set to `application/json`.

Method Signature:

- `public Response createItem(Item item)`: This defines the method signature.
- `public`: Access modifier for the method, making it accessible from outside the class.
- `Response`: The return type of the method is `Response`, an object used to build and return HTTP responses in JAX-RS.
- `createItem`: The method name, indicating its purpose (creating an item).
- `Item item`: The method takes a single argument of type `Item`. This parameter will hold the data for the new item being created, presumably deserialized from the JSON request body by Jersey.

Method Body (Current Implementation):

- `//item.setId(itemList.size() + 1);`: This commented line would have set the ID of the new item based on the current size of the `itemList`. However, it's commented out, so it's not currently used.
- `//itemList.add(item);`: This commented line would have added the new `item` object to the `itemList`. This is also commented out, so the actual creation of the item is not implemented yet.
- `System.out.println(item);`: This line prints the `item` object to the console for debugging purposes. You might want to remove this in a production environment.
- `return Response.status(Response.Status.CREATED).entity("Item created successfully").build();`: This line returns a pre-built `Response` object. Here's what it does:
 - `Response.status(Response.Status.CREATED)`: Sets the HTTP status code of the response to `201 Created`, indicating successful creation.
 - `.entity("Item created successfully")`: Sets the entity (body) of the response to a string message.
 - `.build()`: Builds the final `Response` object.

Step6. Create method to handle GET request to send all Items

```
@GET
@Produces(MediaType.APPLICATION_JSON)
public Response.getItems() {
    return Response.ok(itemList).build();
}
```

Explanation:

The provided code snippet defines a JAX-RS resource method named `getItems` within the `ItemResource` class. Let's break down what it does:

Annotation:

- `@GET`: This annotation indicates that this method handles HTTP GET requests. Typically used for retrieving resources.
- `@Produces(MediaType.APPLICATION_JSON)`: This annotation specifies that the method will produce a response in JSON format. It tells Jersey that the response will have the `Content-Type` header set to `application/json`.

Method Signature:

- `public Response getItems()`: This defines the method signature.
- `public`: Access modifier for the method, making it accessible from outside the class.
- `Response`: The return type of the method is `Response`, an object used to build and return HTTP responses in JAX-RS.
- `getItems`: The method name, indicating its purpose (getting a list of items).
- The method doesn't take any arguments as it's retrieving the entire list of items.

Method Body:

- `return Response.ok(itemList).build();`: This line returns a pre-built `Response` object. Here's what it does:
- `Response.ok(itemList)`: Sets the HTTP status code of the response to `200 OK` and the entity (body) to the `itemList`. This assumes the `itemList` member variable holds the collection of items to be returned.
- `.build()`: Builds the final `Response` object.

Step7. Create method to handle GET request to send single Item by ID

```
@GET
@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public Response getItemById(@PathParam("id") int id) {

    for (Item item : itemList) {
        if (item.getId() == id) {
            return Response.ok(item).build();
        }
    }
    return Response.status(Response.Status.NOT_FOUND).build();
}
```

Explanation:

The code snippet defines a JAX-RS resource method named `getItemById` within the `ItemResource` class.

Let's break down what it does:

Annotations:

- `@GET`: This annotation indicates that this method handles HTTP GET requests. Typically used for retrieving resources.
- `@Path("/{id}")`: This annotation defines a path template for this method. It includes a placeholder `{id}` which will be replaced with a path parameter value during request routing.
- `@Produces(MediaType.APPLICATION_JSON)`: This annotation specifies that the method will produce a response in JSON format. It tells Jersey that the response will have the `Content-Type` header set to `application/json`.

Method Signature:

- `public Response getItemById(@PathParam("id") int id)`: This defines the method signature.
- `public`: Access modifier for the method, making it accessible from outside the class.
- `Response`: The return type of the method is `Response`, an object used to build and return HTTP responses in JAX-RS.
- `getItemById`: The method name, indicating its purpose (getting an item by ID).

- `@PathParam("id") int id`: This parameter retrieves the value from the path parameter named "id". Jersey will automatically extract the value from the URI and convert it to an `int` for this method.

Method Body:

- `for (Item item : itemList) { ... }`: This loop iterates over each `Item` object in the `itemList` member variable.
- `if (item.getId() == id) { ... }`: Inside the loop, it checks if the current item's ID matches the provided `id` parameter.
- `return Response.ok(item).build();`: If a matching item is found, it returns a pre-built `Response` object with:
- `Response.ok(item)`: Sets the HTTP status code to 200 OK and the entity (body) to the matching `item` object.
- `return Response.status(Response.Status.NOT_FOUND).build();`: If the loop iterates through the entire list without finding a match, it returns a pre-built `Response` object with:
- `Response.status(Response.Status.NOT_FOUND)`: Sets the HTTP status code to 404 Not Found to indicate the item with the requested ID wasn't found.

Step8. Create Method to handle PUT request to update the Item

```
@PUT
@Path("/{id}")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response updateItem(@PathParam("id") int id, Item updatedItem) {

    System.out.println(updatedItem.getName());
    for (Item item : itemList) {
        if (item.getId() == id) {
            item.setName(updatedItem.getName());
            return Response.ok("Item updated successfully").build();
        }
    }
    return Response.status(Response.Status.NOT_FOUND).entity("Item not found").build();
}
```

Explanation:

The code snippet defines a JAX-RS resource method named `updateItem` within the `ItemResource` class.

Let's break down what it does:

Annotations:

- `@PUT`: This annotation indicates that this method handles HTTP PUT requests. Typically used for updating existing resources.
- `@Path("/{id}")`: This annotation defines a path template for this method, similar to `getItemById`. It includes a placeholder `{id}`.
- `@Consumes(MediaType.APPLICATION_JSON)`: This annotation specifies that the method expects JSON data in the request body containing the updated item information.
- `@Produces(MediaType.APPLICATION_JSON)`: This annotation specifies that the method will produce a response in JSON format.

Method Signature:

- `public Response updateItem(@PathParam("id") int id, Item updatedItem)`: This defines the method signature.
- `public`: Access modifier for the method, making it accessible from outside the class.
- `Response`: The return type of the method is `Response`, an object used to build and return HTTP responses in JAX-RS.
- `updateItem`: The method name, indicating its purpose (updating an item).

- `@PathParam("id") int id`: This parameter retrieves the value from the path parameter named "id".
- `Item updatedItem`: This parameter holds the data for the updated item, presumably deserialized from the JSON request body by Jersey.

Method Body:

- `System.out.println(updatedItem.getName());`: This line prints the name of the updated item object to the console for debugging purposes. You might remove this in production.
- `for (Item item : itemList) { ... }`: This loop iterates over each `Item` object in the `itemList` member variable.
- `if (item.getId() == id) { ... }`: Inside the loop, it checks if the current item's ID matches the provided `id` parameter.
- `item.setName(updatedItem.getName());`: If a matching item is found, it updates the name of the item in the list with the name from the `updatedItem` object.
- `return Response.ok("Item updated successfully").build();`: It returns a pre-built `Response` object with:
- `Response.ok("Item updated successfully")`: Sets the HTTP status code to 200 OK and the entity (body) to a success message.
- `return Response.status(Response.Status.NOT_FOUND).entity("Item not found").build();`: If the loop finishes without finding a match, it returns a pre-built `Response` object with:
- `Response.status(Response.Status.NOT_FOUND)`: Sets the HTTP status code to 404 Not Found to indicate the item with the requested ID wasn't found.
- `.entity("Item not found")`: Sets the entity (body) to an error message.

Step9. Create method to handle DELETE request to delete the Item

```
@DELETE
@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public Response deleteItem(@PathParam("id") int id) {

    System.out.println("Id to be delete is : "+ id);
    Item itemToRemove = null;
    for (Item item : itemList) {
        if (item.getId() == id) {
            itemToRemove = item;
            break;
        }
    }

    if (itemToRemove != null) {
        itemList.remove(itemToRemove);
        return Response.ok("Item deleted successfully").build();
    } else {
        return Response.status(Response.Status.NOT_FOUND).entity("Item not found").build();
    }
}
```

Explanation:

The code snippet defines a JAX-RS resource method named `deleteItem` within the `ItemResource` class.

Let's break down what it does:

Annotations:

- `@DELETE`: This annotation indicates that this method handles HTTP DELETE requests. Typically used for deleting resources.
- `@Path("/{id}")`: This annotation defines a path template for this method, similar to `getItemById` and `updateItem`. It includes a placeholder `{id}`.
- `@Produces(MediaType.APPLICATION_JSON)`: This annotation specifies that the method will produce a response in JSON format.

Method Signature:

- `public Response deleteItem(@PathParam("id") int id)`: This defines the method signature.
- `public`: Access modifier for the method, making it accessible from outside the class.
- `Response`: The return type of the method is `Response`, an object used to build and return HTTP responses in JAX-RS.
- `deleteItem`: The method name, indicating its purpose (deleting an item).

- `@PathParam("id") int id`: This parameter retrieves the value from the path parameter named "id".

Method Body:

- `System.out.println("Id to be deleted is : "+ id);`: This line prints the ID to be deleted for debugging purposes. You might remove it in production.
- `Item itemToRemove = null;`: This line declares a variable `itemToRemove` of type `Item` and initializes it to `null`. It will hold the item to be deleted if found.
- `for (Item item : itemList) { ... }`: This loop iterates over each `Item` object in the `itemList` member variable.
- `if (item.getId() == id) { ... }`: Inside the loop, it checks if the current item's ID matches the provided `id` parameter.
- `itemToRemove = item;`: If a matching item is found, it assigns the item to the `itemToRemove` variable.
- `break;`: The `break` statement exits the loop after finding the first matching item.
- `if (itemToRemove != null) { ... } else { ... }`: This conditional block checks if a matching item was found.
- `itemList.remove(itemToRemove);`: If an item was found (`itemToRemove` is not `null`), it removes the item from the `itemList`.
- `return Response.ok("Item deleted successfully").build();`: It returns a pre-built `Response` object with:
 - `Response.ok("Item deleted successfully")`: Sets the HTTP status code to 200 OK and the entity (body) to a success message.
- `else:` If no matching item was found (`itemToRemove` is `null`).
- `return Response.status(Response.Status.NOT_FOUND).entity("Item not found").build();`: It returns a pre-built `Response` object with:
 - `Response.status(Response.Status.NOT_FOUND)`: Sets the HTTP status code to 404 Not Found to indicate the item with the requested ID wasn't found.
 - `.entity("Item not found")`: Sets the entity (body) to an error message.